

A Physics-Informed Generative Adversarial Network for Advancing Solutions in Ocean Acoustics

Rui Xia,^{1,2} Xiao-Wei Guo,^{1,2, a)} Huajian Zhang,^{1,2} Genglin Li,^{1,2} Jing Xiao,^{1,2} Qisong Xiao,^{1,2} Min Song,^{1,2}
Chao Li,^{1,2} and Jie Liu^{3,2,1}

¹⁾Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology

²⁾College of Computer Science and Technology, National University of Defense Technology

³⁾National Key Laboratory of Parallel and Distributed Computing, National University of Defense Technology

(*Electronic mail: guoxiaowei@nudt.edu.cn)

Advancements in artificial intelligence, notably the groundbreaking efforts in deep learning exemplified by physics-informed neural networks, have opened up innovative pathways for addressing intricate ocean acoustic problems. However, conventional physics-informed neural networks are limited in solving high-frequency forward and inverse problems. This paper introduces a novel physics-informed generative adversarial network integrating a forward-solving network (generator) and an inverse parameter-estimating network (discriminator). The generator network incorporates convolutional neural networks with hard-constrained boundary conditions and optimized loss functions to effectively predict the solution governed by the time-domain wave equation. For inverse problems, a discriminator is introduced for parameter estimation to complete the generative adversarial network. Furthermore, customized optimization strategies and an adaptive weighting loss function are devised to boost the training performance further. The test results of both forward and reverse cases show the advantage of our model over existing methods in terms of accuracy. The result indicates its vast potential for applications in ocean acoustics engineering.

I. INTRODUCTION

Owing to the capabilities of traversing vast distances in the marine environment and piercing through various underwater obstacles, sound waves are a paramount tool for communication and detection in the ocean. Computational acoustics, as discussed in¹, delves into the intricate characteristics and governing principles of underwater sound propagation within ocean waveguides and explores methods to investigate the ocean utilizing acoustic waves. Advancements in ocean acoustic technologies have not only enriched our understanding of the ocean's architecture² and facilitated monitoring of oceanic climates³, but they have also significantly bolstered the progress of other research disciplines about the ocean⁴.

Various approaches have been proposed for numerically solving the wave equation, each suited to specific propagation conditions. Normal mode theory, which decomposes the acoustic field into a series of normal modes, is especially effective for low-frequency propagation in shallow water environments. Recent work in this area, such as that by Luo et al.⁵, introduces a stepwise coupled-mode model using the direct global matrix approach, improving the efficiency of solving time-domain wave equations. Additionally, recent advancements in normal mode theory have been explored to further improve accuracy, as seen in⁶ and adiabatic mode models^{7,8}.

In contrast, high-frequency acoustic propagation is often modeled using ray methods, which rely on geometric acoustics to describe sound paths. Although these methods are computationally efficient, they face challenges when applied to complex environments. Recent adaptive algorithms are being developed to address these issues⁹. The parabolic equation (PE) method strikes a balance between accuracy and

computational cost, making it especially suitable for mid-to-long-range propagation scenarios. Recent advancements in PE have further enhanced its applicability to broadband problems¹⁰. Numerical approaches such as the Finite Difference Method (FDM), Finite Volume Method (FVM), and Finite Element Method (FEM) offer high accuracy, but they tend to be computationally intensive. For example, Jung Baek-Ho et al.¹¹ proposed the Marching-On in Degree Finite Difference Method (MOD-FDM), while Xia et al.¹² applied FVM within a general Computational Fluid Dynamics (CFD) framework. Recent research has focused on hybrid methods to improve efficiency while maintaining the accuracy of these models¹³⁻¹⁵.

Although numerical methods have made significant progress in ocean acoustic research and applications, they still face many challenges in underwater acoustics calculations in large-scale and complex marine environments due to their high computational requirements and weak tolerance for noise. Deep learning methods have brought new perspectives to solving partial differential equations (PDEs), such as the wave equation. Due to its physical-driven nature, the Physics-Informed Neural Networks (PINNs) proposed by Raissi Maziar et al.¹⁶ have quickly become a research hotspot.

Recent advancements in Physics-Informed Neural Networks (PINNs) have primarily focused on improving computational accuracy and expanding their application to solve direct and inverse problems governed by partial differential equations (PDEs). Notable developments include the Bayesian PINN (B-PINN) proposed by Yang et al.¹⁷, which integrates Bayesian methods to handle noisy data in PDE problems, and the HP-PINN framework by Ehsan Kharazmi et al.¹⁸, which enhances accuracy using nonlinear approximations and hp refinement techniques. Zhang et al.¹⁹ designed PINN-FFHT to address fluid flow and heat transfer challenges through innovative boundary conditions, while Yan et al.²⁰ introduced ST-PINN, a self-training algorithm for improved learning. Additionally, Xu et al.²¹ applied

^{a)}Xiao-Wei Guo is the corresponding author of this paper.

multi-task learning to increase training efficiency for elastic and hyperelastic inverse problems. Beyond these advancements, PINNs have shown significant promise in applications such as nano-optics, metamaterials design²², ultrasonic crack quantification²³, wave propagation²⁴, and geological research²⁵. Furthermore, PINN's integration with other deep learning architectures like CNNs, GCNs, and LSTMs^{26,27,28,29,30} has demonstrated the versatility of physical information-driven models in tackling a wide range of real-world problems, including those in ocean acoustics³¹.

Existing physics-informed models struggle with computational accuracy, time, and convergence stability, limiting their application to complex ocean acoustics problems. To address this, we apply physics-informed Generative Adversarial Networks (GAN)³² to design a framework that estimates both the sound speed distribution and corresponding time-domain acoustic fields³³. Through adversarial training between the forward solution and parameter evaluation networks, we enhance their performance, enabling precise sound speed profiles and detailed acoustic field reconstructions—providing a novel solution to ocean acoustics challenges.

In this paper, we propose PGONet, a Physics-Informed Generative Adversarial Network for Advancing Solutions in Ocean Acoustics, and verify its solution accuracy on forward and inverse tasks with wave equation constrained cases. The main contributions of this paper are summarised as follows.

- We propose a novel physics-informed GAN architecture combining a forward-solving network (generator) and an inverse parameter network (discriminator). Driven by the acoustics wave equation, this GAN efficiently solves typical ocean acoustic forward and inverse problems.
- We present an efficient approach for learning spatiotemporal physical information of the wave equation. Using a CNN layer for spatial discretization, integrating temporal discretization formats, and enforcing boundary conditions with hard constraints, our network outperforms existing methods in prediction accuracy.
- To solve the inverse problem, we train PGONet using a GAN-based approach and adopt a customized optimization strategy and adaptive weighted loss function to improve performance. After testing on solving the acoustic inverse problem, PGONet shows high accuracy in the ocean acoustic inverse problem.

The remainder of the paper is structured as follows: Section II will outline the architecture of PGONet and the training algorithm. In Section III, we will present the experimental cases and the verification results, then provide an analysis based on the results. Section IV will summarize the conclusions. The accompanying code for this study has been uploaded to <https://github.com/DongFengZero/PGONet>.

II. METHOD

The focus of our study in this paper is to solve the forward and inverse problems of the wave equation in the time domain. The equation form can be expressed as Eq.(1). P represents the sound pressure, and c represents the speed of sound in the waveguides.

$$\nabla^2 P - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0 \quad (1)$$

Many types of forward and inverse problems are governed by Eq.(1). The **forward problem** in ocean acoustics refers to solving the wave equation with given conditions to predict the propagation characteristics of waves in a given medium. For the cases studied in this paper, the given conditions are the sound speed c , the boundary conditions, and what needs to be predicted is the time-domain distribution P of the entire sound field. For the **inverse problem**, a small number of reference solutions P_{ref} on the entire time-domain distribution of the sound field are given to infer the sound speed distribution c of the sound field, to infer the boundary configuration or obstacle locations based on the inferred sound speed distribution.

This section will mainly discuss how we solve these two problems with PGONet. We will introduce our work from two main perspectives: the neural network architecture and the training algorithms of PGONet.

A. Architecture of PGONet

PGONet is a novel physics-informed GAN framework integrating a forward-solving network (generator) and an inverse parameter-estimating network (discriminator).

The parameter estimating network is shown in FIG. 1. The parameter estimation network acts as the discriminator in the GAN network. The sound speed $\hat{c}$ it evaluates will be used as the sound speed c used by the generator (forward solution network) to calculate the sound field in the next iteration. The network estimates the sound velocity distribution of the generated sound field by inputting a generated sound field of a generator until it meets the shutdown criterion. These sound field distributions may not be accurate or contain a lot of noise. Still, it helps the discriminator generate sound speed distributions close to the actual or new sound from random noise data. At the same time, we introduce the reference information of the actual sound field into its loss function to guide its sound velocity distribution to be closer to the sound velocity distribution in the natural environment. We will introduce the loss function in the training algorithm section.

The forward-solving network acts as a generator in the GAN paradigm in FIG. 2, simulating acoustic propagation based on an initial guess of the sound velocity distribution. This network can efficiently calculate the expected time-domain sound field distribution given the velocity field. Different from introducing the residual term of the partial differential equation into the loss function, we design an optimization module with physical information to guide the conver-

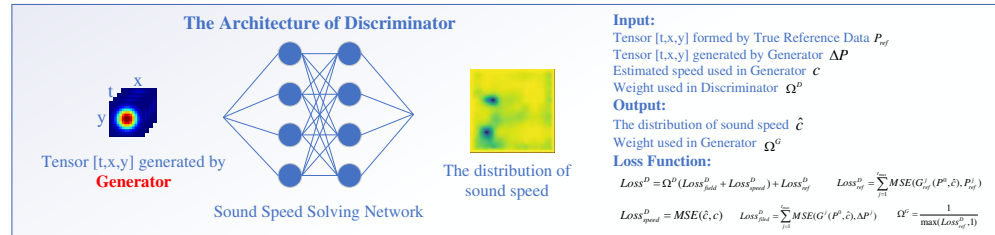


FIG. 1. The architecture of discriminator. The discriminator in GAN estimates the inverse parameters. It takes sound field data from the forward solution network as input and outputs the corresponding sound speed distribution. Real reference solutions are introduced during inverse problem-solving to guide the network in generating accurate sound speed distributions.

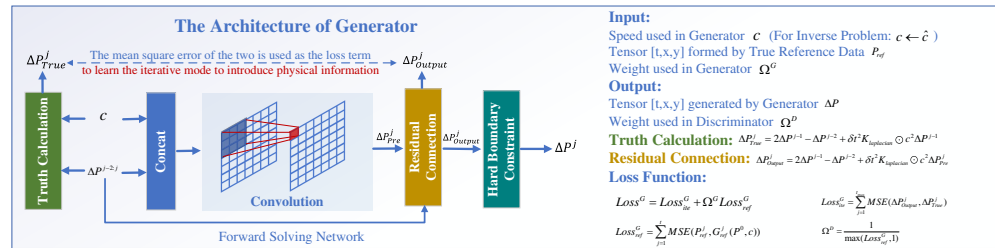


FIG. 2. The architecture of generator. The generator in GAN acts as a forward iterative solver to calculate the sound speed distribution and water configuration of a given underwater sound field. It iteratively calculates the value through the truth calculation module, and then minimizes the mean square error using the Concat, convolution, and residual connection modules. In the reverse solving task, it generates the time domain sound field distribution with the predicted sound speed and helps the discriminator refine the sound speed distribution by adding noise.

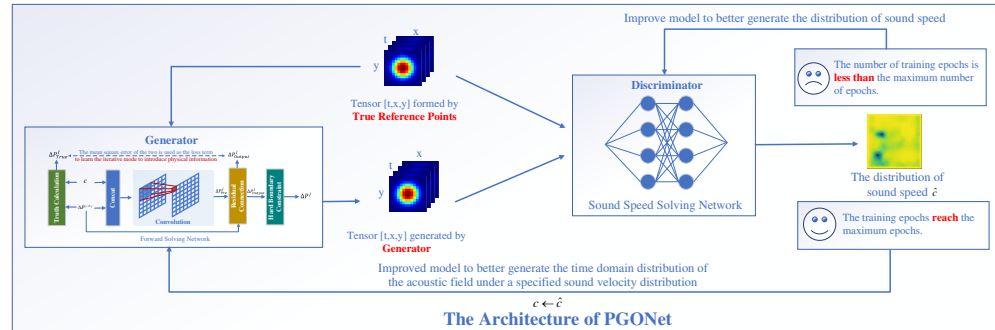


FIG. 3. The architecture of PGONet. The PGONet adversarial network consists of a discriminator and a generator. The discriminator estimates the sound speed from the generator's distribution, guiding it toward the real sound speed with the help of real solutions. The generator produces the sound field distribution based on the discriminator's estimate, fitting the real sound field. The generator's noisy output also helps the discriminator refine its estimate, avoiding local optima. Both networks alternate learning, stopping when the maximum training rounds are reached.

Parameter	Meaning of Parameter
c	Sound speed used in generator
$\hat{c}$	Sound speed predicted by discriminator
ΔP^j	The sound field tensor calculated by the generator of PGONet at j -th timestep (ΔP represents the value at all time steps)
P^0	The value of the initial state sound field
P_{ref}^j	Tensor consisting of true reference points in the sound field at j -th timestep (P_{ref} represents the value at all time steps)
$G^j(P^0, c)$	The sound field value at j -th timestep calculated by the generator with initial state P^0 and sound speed c ($G_{ref}^j(P^0, c)$ represents the value at the true reference points at j -th timestep)
MSE	Mean square error calculation function
t_{max}	The maximum number of time steps.

TABLE I. Table of the meanings of the parameters in Eq.(2)-Eq.(28).

gence of the neural network, including the actual value calculation module and the residual connection module.

We first introduce the **truth calculation module**. In each prediction step, the input includes the sound field distribution and sound speed of the previous two predicted steps. We can use this information to calculate a value through the difference form of the wave equation (Eq.(3)). This module introduces the physics of the wave equation into the network in the form of differences in the loss function, thereby guiding the model to convergence. For symbols not specifically explained in the section II, we have listed their meanings in Table I. δ_t , δ_x , δ_y represent the time step and the grid size in the x and y directions. $K_{laplacian}$ represents the Laplace convolution kernel, as shown in Eq.(2). $\odot$ is the convolution operation.

$$K_{laplacian} = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \times \frac{1}{\delta_x^2} + \begin{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \end{bmatrix} \times \frac{1}{\delta_y^2} \quad (2)$$

$$\Delta P_{True}^j = 2\Delta P^{j-1} - \Delta P^{j-2} + \delta t^2 K_{laplacian} \odot c^2 \Delta P^{j-1} \quad (3)$$

Through the calculation of the convolution neural network(The model used in this article is a single-layer convolution.), a preliminary prediction ΔP_{Pre}^j for the current time step is generated. The calculation process can be expressed as Eq.(4). $\Delta P^{j-2:j}$ is consistent with the syntax of Python code. That is, it includes ΔP^{j-2} , ΔP^{j-1} , but does not include ΔP^j . *Concat* means tensor concatenation, and *Conv* means a single convolution layer.

$$\Delta P_{Pre}^j = Conv(Concat(\Delta P^{j-2:j}, c)) \quad (4)$$

Inspired by the work of³⁴ et al., we designed a **residual connection module** with a backward difference format. We again use the wave equation's differential form (Eq.(5))

to generate the output. This step is equivalent to changing the central difference in Eq.(3) to the forward difference in Eq.(5). Similarly, this module also introduces the physics of the wave equation into the network in the form of differences and guides the model to converge by fitting the difference form.

$$\Delta P_{Output}^j = 2\Delta P^{j-1} - \Delta P^{j-2} + \delta t^2 K_{laplacian} \odot c^2 \Delta P_{Pre}^j \quad (5)$$

Finally, we introduce the treatment of boundary conditions. In recent years, many studies have focused on designing physical information embedding neural networks on boundary conditions and achieved good results. N. Sukumar et al.³⁵ proposed the precise imposition of boundary conditions using distance functions. Wang et al.³⁶ implement exact Dirichlet boundary conditions and enable efficient simulation of solid mechanics problems. We refer to the hard constraints that have been widely used³⁷⁻⁴⁰, that is, after generating the sound field distribution, interpolation is performed through physical formulas to force the corresponding boundary conditions to be satisfied. Hard constraint ensures the stability of the boundary conditions of the solution domain.

Using the following numerical difference method, we enforce hard constraints at the boundary. We developed three boundary conditions commonly used in hydroacoustics, namely rigid boundary, pressure-release boundary, and non-reflecting boundary, which are shown in Eq.(6), Eq.(7), Eq.(8), respectively¹².

(i) The pressure-release boundary

$$P = 0 \quad (6)$$

(ii) The rigid boundary

$$\frac{\partial P}{\partial \mathbf{n}} = 0 \quad (7)$$

(iii) The non-reflecting boundary

$$\frac{\partial P}{\partial t} + \nabla(cP) = 0 \quad (8)$$

First, we analyze the pressure-release boundary. Since the pressure value on the pressure-release boundary is constant at 0, we only need to hard constrain the boundary condition to 0 as shown in Eq.(9).

(i) The pressure-release boundary

$$P = 0 \quad (9)$$

Secondly, we design the boundary based on the convolution kernel representation of rigid boundary and non-reflecting boundary. Eq.(10)-Eq.(14) represent the convolution kernels K_{dt} , K_{d2t} , K_{dx} , K_{dy} , $K_{laplacian}$ and $K_{gradient}$ for calculating the first-order time derivative, the second-order time derivative, first-order x -direction guide, first-order y -direction guide, Laplace operator and gradient operator, respectively. For the convenience of calculation, it is assumed that each grid has only two calculation directions of x and y .

$$K_{dt} = \begin{bmatrix} [-1], [0], [1] \end{bmatrix} \times \frac{1}{2\delta_t} \quad (10)$$

$$K_{d2t} = \begin{bmatrix} [1], [-2], [1] \end{bmatrix} \times \frac{1}{\delta_t^2} \quad (11)$$

$$K_{dx} = \begin{bmatrix} [-1] \ 1 \end{bmatrix} \times \frac{1}{\delta_x} \quad (12)$$

$$K_{dy} = \begin{bmatrix} [-1] \\ 1 \end{bmatrix} \times \frac{1}{\delta_y} \quad (13)$$

$$K_{gradient} = \begin{bmatrix} [-1] \ 1 \end{bmatrix} \times \frac{1}{\delta_x} + \begin{bmatrix} [-1] \\ 1 \end{bmatrix} \times \frac{1}{\delta_y} \quad (14)$$

We can modify the expression to Eq.(15)-Eq.(16), Eq. (17), and use the equation to update the boundary of the solution domain at the end of each time step iteration. In summary, we implement hard constraints through Eq.(9), Eq.(15)-Eq.(16), Eq. (17) so that the solution meets the boundary conditions.

(ii) The rigid boundary along x and y directions

$$K_{dx} \odot P = 0 \quad (15)$$

$$K_{dy} \odot P = 0 \quad (16)$$

(iii) The non-reflecting boundary along the 45-degree wedge

$$K_{dt} \odot P + K_{gradient} \odot cP = 0 \quad (17)$$

In summary, we have formed a complete adversarial network PGONet, the specific structure shown in FIG. 3. The PGONet adversarial generative network comprises a discriminator and a generator. The discriminator predicts a sound speed for the sound field, drawing from the sound speed distribution produced by the generator. It then steers this estimate towards the genuine sound speed distribution, with assistance from actual measurements. Concurrently, the generator constructs the sound field distribution based on the discriminator's sound speed predictions and adapts to the real sound field, guided by actual data. During the learning process, incorporating sound field distributions with random noise can provide feedback to the discriminator, enabling it to generate more realistic sound speed distributions or explore new distributions to evade local optima. The two networks compete and learn until predefined termination criteria (e.g., reaching a training round) are met.

B. Training Algorithm

In this section, we will introduce the specific training algorithm. As a physics-driven neural network, PGONet learns a reusable iterative solver during the solution process. The iterative solver only needs to master the iteration pattern during the solution process so that it can be repeatedly applied to other physical scenarios that follow the same equation. Next, we take the loss function of the generator at the j -th time step as an example for analysis. We first give the overall loss function of the generator as Eq.(18).

$$Loss^G = Loss_{ite}^G + Loss_{ref}^G \quad (18)$$

Although it shares the same physics-driven perspective with PINN, it is not entirely about calculating residuals. It can learn this fixed iteration pattern on accurate or inaccurate data to master the iteration pattern. Regardless of whether the forward time step is correct, the true value calculation formula (Eq.(3)) can always calculate the expected value that should be obtained under the iteration mode based on the forward time step. We take the mean square error of the final value after the residual connection module (Eq.(5)) and the expected value and use the mean square error as a part of the loss function as Eq.(19). As a result, we can learn physics-driven iteration patterns more efficiently and with better convergence.

$$Loss_{ite}^G = \sum_{j=1}^{t_{max}} MSE(\Delta P_{Output}^j, \Delta P_{True}^j) \quad (19)$$

In fact, for the forward solution task, it is only necessary to use $Loss_{ite}^G$ as the loss function. For the inverse task, we also introduce a small amount of known reference true solutions in the forward solution process to accelerate the convergence of the model. The formula is shown in Eq.(20). The meaning of ΔP_{ref}^j and P_{ref}^j is in Table I.

$$Loss_{ref}^G = \sum_{j=1}^I MSE(P_{ref}^j, G_{ref}^j(P^0, c)) \quad (20)$$

In contrast, the sound speed-solving network is similar to the discriminator, evaluating the generated sound field velocity distribution and distinguishing between physically reasonable sound fields and those that deviate from realistic behavior. In the inverse task of estimating the speed $\hat{c}$, we take advantage of the limited sensor detection data. The overall loss function can be written as Eq.(21).

$$Loss^D = Loss_{field}^D + Loss_{ref}^D + Loss_{speed}^D \quad (21)$$

Unlike the generator, which solves the problem timestep by timestep, the discriminator solves the problem in batches consisting of t_{max} timesteps. We have three loss terms: Eq.(22)-Eq.(24). Eq.(22) introduces a small number of reference true solutions to assist the model convergence. Eq.(23) is used to

assist it in correctly predicting the sound **speed** of the sound field based on the sound field value. Eq.(24) is used to introduce the sound **field** data generated by the generator under the predicted sound speed $\hat{c}$ so that the predicted sound speed given by the discriminator is close to the sound speed c used by the generator, thereby improving the performance of the model discriminator in predicting the sound speed and promoting its convergence. $G^j(P^0, \hat{c})$ represents the sound field value at time j calculated by the generator with an initial state of P^0 and a sound speed of $\hat{c}$. The meaning of $G_{ref}^j(P^0, \hat{c})$ is in Table I.

$$Loss_{ref}^D = \sum_{j=1}^{t_{max}} MSE(G_{ref}^j(P^0, \hat{c}), P_{ref}^j) \quad (22)$$

$$Loss_{speed}^D = MSE(\hat{c}, c) \quad (23)$$

$$Loss_{field}^D = \sum_{j=1}^{t_{max}} MSE(G^j(P^0, \hat{c}), \Delta P^j) \quad (24)$$

Driven by the objective function, the two networks interact, causing the forward solver to produce increasingly realistic sound fields. In contrast, the parameter evaluation network becomes better at detecting differences, resulting in a refinement process that converges to a realistic sound speed distribution. PGONet's adoption of this adversarial training strategy streamlines the traditionally complex and computationally intensive process of inverse acoustic modeling. It enables the system to generalize to unseen conditions, broadening its applicability across diverse underwater acoustic scenarios.

Next, we need to determine the weights in the loss function. First, we use the parameter estimation network as a discriminator. The main loss terms of the parameter estimation network are the partial differential residual and the sound speed discrimination loss terms. A loss term also introduces reference information into the parameter estimation network. However, when the solution accuracy of the forward solution network is low, the solution information of the forward solution network will not bring much benefit to the parameter estimation network. At this time, we introduce the weight Ω^D to encourage the parameter estimation network to focus on the reference true solution. When the solution accuracy of the forward solution network is improved, we dynamically increase the weight of this loss term so that this loss term can fit the solution information of the forward solution network to obtain more information.

Similarly, when the discriminator's estimation accuracy of the sound speed is low, we encourage the generator to focus on generating the sound field at the corresponding sound speed. When the discriminator's accuracy gradually improves, the weight of the reference information loss term Ω^G is dynamically increased to encourage the generator to generate a sound field closer to the real one.

We propose the adaptive weighting loss function to capture the dynamic training characteristics, as shown in Eq.(25)-Eq.(26). Other unspecified weights are 1.

Algorithm1 PGONet Training

Input:

Initial state sound field P^0 ,
Maximum epochs per time step used in generator $E_{poch}^G_{max}$,
Maximum epochs at first time step $E_{poch}^0_{max}$,
Maximum epochs used in discriminator $E_{poch}^D_{max}$,
Tensor consisting of true reference points P_{ref} ,
The maximum error that can be tolerated ϵ ,
Estimated sound field ΔP ,
Alternating training epochs $Iter_{max}$

Output:

Estimated sound speed $\hat{c}$, Estimated sound field ΔP

- 1: Initialization of the generator and the discriminator
- 2: $\Omega^G = 1e8, \Omega^D = 1e8$
- 3: Randomly generate a sound speed c
- 4: **for** $E_{poch} = 0; E_{poch} < Iter_{max}; E_{poch} + 1$ **do**
- 5: $\Delta P, \Omega^D = \text{Generator}(P^0, c, E_{poch}^G_{max}, E_{poch}^0_{max}, P_{ref}, \epsilon, \Omega^G)$
- 6: $\hat{c}, \Omega^G = \text{Discriminator}(\Delta P, c, E_{poch}^D_{max}, P_{ref}, \Omega^D)$
- 7: $c \leftarrow \hat{c}$
- 8: **return** $\Delta P, \hat{c}$

$$\Omega^G = \frac{1}{\max(Loss_{ref}^D, 1)} \quad (25)$$

$$\Omega^D = \frac{1}{\max(Loss_{speed}^D, 1)} \quad (26)$$

Then the loss functions corresponding to the two networks can be rewritten as Eq.(27)-Eq.(28).

$$Loss^D = \Omega^D (Loss_{field}^D + Loss_{speed}^D) + Loss_{ref}^D \quad (27)$$

$$Loss^G = Loss_{ite}^G + \Omega^G Loss_{ref}^G \quad (28)$$

From the perspective of the iterative solution, a significant source of error is the propagation error along the time dimension. As the time step advances, the mistake in the forward time step is gradually amplified. This effect will seriously affect the convergence efficiency and solution accuracy of the network, and the longer the solution time series, the more noticeable this error is. To suppress this effect, we further optimized the loss function proposed previously and proposed a training algorithm based on the optimized loss function.

The PGONet training algorithm is shown in Algorithm 1. As shown in FIG. 3 and Algorithm 1, the generator and discriminator are trained alternately within the specified number of iterations. The generator generates a new sound field ΔP for the discriminator, and the discriminator gives the generator a new sound speed estimate $\hat{c}$. The estimation of the sound field is done during the adversarial training process. Ω^G and Ω^D represent the loss weights of the generator and the discriminator, respectively.

Algorithm2 Generator Training (For Forward Problem)

Input:
Initial state sound field P^0 ,
Speed of sound c ,
Maximum epochs per time step used in generator $Epoch_{max}^G$,
Maximum epochs at first time step $Epoch_{max}^0$,
The maximum error that can be tolerated ϵ
Output:
Estimated sound field ΔP

```

1: Initialization of the generator network
2:  $\Delta P^1 = \Delta P^0 = P^0, j = 0$ 
3: for  $j = 0; j < t_{max}; j++ = 1$  do
4:    $Epoch = 0$ 
5:   while ( $Epoch < Epoch_{max}^G$  and  $j! = 0$ ) or ( $Epoch < Epoch_{max}^0$  and  $j == 0$ ) do
6:      $\Delta P^{j+2} = \text{Generator}(\Delta P^{j:j+2}, c)$ 
7:      $Loss_{ite}^G = \text{ComputeGenLoss1}(\Delta P^{j:j+3}, c)$ 
8:     if  $Loss_{ite}^G < \epsilon$  and  $j! = 0$  then
9:       break
10:     $Loss_{ite}^G \leftarrow \text{backward}()$ 
11:     $Epoch = Epoch + 1$ 
12:  $\Delta P \leftarrow \Delta P^{2:j+2}$ 
13: return  $\Delta P$ 

```

The training of the generator for the forward problem is shown in Algorithm 2. The training process of the generator is trained forward in time-series order, using the data of the first two time steps ΔP^{j-2} and ΔP^{j-1} to calculate ΔP^j . Each epoch corresponds to a round of the optimization process that predicts one timestep. We will calculate the next time step for each time step when the convergence error exceeds the threshold or has reached $Epoch_{max}^G$. In addition, we also require that in the calculation of the first time step, regardless of whether the threshold is met, the training round must be forced to meet $Epoch_{max}^0$, and this value must be greater than $Epoch_{max}^G$ of other time steps. The purpose is to enable it to be fully trained in the first step to save the cost of subsequent training. The ComputeGenLoss1 function in Algorithm 2 is calculated concerning Eq.(19). The syntax of $\Delta P^{j:j+3}$ is consistent with $\Delta P^{j-2:j}$ in Eq.(4).

The training of the generator for the inverse problem is shown in the Algorithm 3. Compared with the forward-solving algorithm, the difference is that it calculates more loss terms, uses an adaptive weight loss function, and generates new weights for the discriminator. The ComputeGenLoss2 function in Algorithm 3 is calculated concerning Eq.(28). The UpdateDisWeight function in Algorithm 3 is calculated concerning Eq.(26). $Loss_{ref}^G$ is calculated by Eq.(20). $Loss_{ite}^G$ is calculated by Eq.(19).

For the discriminator, the training process is relatively simple. The sound field generated by the generator is input into the linear layer network to generate a sound field distribution. The specific pseudo code is shown in Algorithm 4. The ComputeDisLoss , UpdateGenWeight function in Algorithm 4 is calculated as Eq.(27), Eq.(25), respectively.

We provide the discriminator and generator's loss functions and running codes in the adversarial network PGONet. These two networks will form an adversarial network and will be

Algorithm3 Generator Training (For Inverse Problem)

Input:
Initial state sound field P^0 ,
Speed of sound c ,
Maximum epochs per time step used in Generator $Epoch_{max}^G$,
Maximum epochs at first time step $Epoch_{max}^0$,
Tensor consisting of true reference points P_{ref} ,
The maximum error that can be tolerated ϵ ,
Weight of loss function of generator Ω^G
Output:
Estimated sound field ΔP
Weight of loss function of discriminator Ω^D

```

1:  $\Delta P^1 = \Delta P^0 = P^0$ 
2: for  $j = 0; j < t_{max}; j++ = 1$  do
3:    $Epoch = 0$ 
4:   while ( $Epoch < Epoch_{max}^G$  and  $j! = 0$ ) or ( $Epoch < Epoch_{max}^0$  and  $j == 0$ ) do
5:      $\Delta P^{j+2} = \text{Generator}(\Delta P^{j:j+2}, c)$ 
6:      $Loss^G, Loss_{ref}^G, Loss_{ite}^G = \text{ComputeGenLoss2}(\Delta P^{j:j+3}, c, P_{ref}, \Omega^G)$ 
7:     if  $Loss_{ite}^G < \epsilon$  and  $j! = 0$  then
8:       break
9:      $Loss^G \leftarrow \text{backward}()$ 
10:     $Epoch = Epoch + 1$ 
11:  $\Omega^D = \text{UpdateDisWeight}(Loss_{ref}^G)$ 
12:  $\Delta P \leftarrow \Delta P^{2:j+2}$ 
13: return  $\Delta P, \Omega^D$ 

```

Algorithm4 Discriminator Training

Input:
Sound field generated by generator ΔP ,
Speed of sound used by generator c ,
Maximum epochs used in discriminator $Epoch_{max}^D$,
Tensor consisting of true reference points P_{ref} ,
Weight of loss function of discriminator Ω^D
Output:
Estimated sound speed $\hat{c}$,
Weight of loss function of generator Ω^G

```

1: for  $Epoch = 0; Epoch < Epoch_{max}^D; Epoch++ = 1$  do
2:    $\hat{c} = \text{Discriminator}(\Delta P)$ 
3:    $Loss^D, Loss_{ref}^D = \text{ComputeDisLoss}(\Delta P, c, \hat{c}, P_{ref}, \Omega^D)$ 
4:    $Loss^D \leftarrow \text{backward}()$ 
5:    $\Omega^G = \text{UpdateGenWeight}(Loss_{ref}^D)$ 
6: return  $\hat{c}, \Omega^G$ 

```

optimized alternately in the process shown in FIG. 3. When the discriminator does not reach the specified shutdown standard, it will continue to optimize itself to improve the sound speed prediction level. When the discriminator reaches the shutdown standard, it will output a time series sequence to optimize the generator's ability to predict the sound field distribution. It forms the adversarial structure of PGONet for solving the ocean inversion problem.

It is worth noting that since PGONet is physically driven, no labeled dataset is required during training. Only the boundary conditions and the initial values of the wave equation at the start time are needed to predict the time series of the sound

field after training. Similarly, in solving the inverse problem, the model only takes the sound source, the sound source form, and a few sampling points of the sound field value as input to predict the sound speed distribution at different locations, again without needing additional specific datasets.

III. RESULTS

This chapter mainly discusses the testing results for validating the PGONet. Section III A introduces the forward and inverse problem cases used in the experiments. Section III B provides the experimental platform information and the settings of hyperparameters used in the tests. Section III C and Section III D presents the key results and data for the forward and inverse problem tests. The last section discusses the performance advantages of using PGONet compared to existing methods.

A. Problem Description

In the preface of Section II, we briefly introduced the forward and inverse problems involved in this paper. In this section, we introduce the issues involved in detail.

1. Forward Problem

The forward-solving tasks involved in this paper are divided into three categories. The first category is **boundary condition test**, the second category is **forward solving test**, and its structure and boundary conditions are shown in FIG. 4(a). The third category is **comparison test**, and its structure and boundary conditions are shown in FIG. 4(d).

The benchmark case for solving was proposed by the Acoustical Society of America (ASA) in the 1980s, with particular emphasis on the wedge-shaped waveguide scenario in^{41,42}, which constitutes the benchmark case for the forward solver tests in this paper. Among the myriad ocean sound propagation paradigms, this case stands out as one of the few that allows an analytical solution. As such, it is the primary benchmark for evaluating the effectiveness of our solver. The wedge configuration shown in FIG. 4 depicts the sea surface as a pressure-release boundary and the bottom boundary as a rigid boundary. The left edge is a non-reflecting boundary without any reflective effects. The wedge's angle is denoted as θ_0 , while the sound source is positioned at (r_s, θ_s) . The boundary specifications for all test cases used for the boundary condition test and the forward solving test are consistent with those shown in FIG. 4(a). When solving the problem in the forward direction, the calculation process of the analytical solution can refer to the work of Luo et al.⁵.

As shown in Table II, the required input for the forward solution case is the sound velocity distribution and initial state, and the output is the time domain distribution of the sound field under the corresponding conditions. It is consistent for

both boundary condition tests, forward solution tests, and comparison tests.

There are some differences between these forward solution tests. The performance analysis test does not give a sound source but provides an initial state of the sound field distribution and asks it to calculate it within a specific time range. The initial state of the test is given by Eq.(29), where (x, y) is the corresponding grid coordinate in the sound field. This coordinate has nothing to do with the grid size. It's just an index.

$$P_0(x, y) = 1000e^{-\frac{(x-64)^2 + (y-64)^2}{100}} \quad (29)$$

Boundary condition testing and forward solution testing require the coordinates and form of the sound source. The boundary condition case uses a square wave sound source, while the forward solution case uses a fixed frequency sine sound source. Due to the difference in the form of the sound source, the boundary condition test and the comparison test do not have an analytical solution. In contrast, the forward solution test case has an analytical solution. The boundary conditions used in the comparison test are inconsistent with the boundary condition test cases and the correct answer test cases. Please refer to Table III for more information about specific cases.

2. Inverse Problem

The inverse problem-solving tasks involved in this paper include two categories. The first category is **Pressure-Release Boundary Detection Problem**, and the second is **Multi-obstacle Detection Problem**. As shown in Table II, the inputs of these two types of inverse tasks are the sampling point location, the actual data of the sampling point, the location of the sound source, and the sound waveform, and the output is the sound velocity distribution of the region. The output sound velocity distribution is used to fit the pressure-release boundary configuration or the location of multiple obstacles. The following two parts will introduce the solution to these two inverse problems and the optimization of data processing.

Pressure-Release Boundary Detection Problem. The inverse task of pressure release boundary detection is to fit the pressure release boundary contour by inverting the resulting sound velocity distribution based on the results of the partial reference solution provided by the sensor. In this task, sensors and sound sources are randomly placed, as shown in FIG. 4(b). The upper and lower boundaries of the water environment are the pressure-release boundaries, and the lower boundary is the boundary to be predicted. The other boundaries are the non-reflecting boundaries. The specific information of the cases used in the experiment is shown in Table V.

In this case, we assume that the bottom boundary to be predicted is a pressure release boundary. The core feature of the pressure relief boundary condition is that the pressure at the boundary is zero ($P = 0$), and the sound speed is zero

($c = 0$) can indirectly achieve this condition, because sound waves cannot propagate in a medium with a sound speed of zero, resulting in zero pressure at the boundary. However, this assumption does not fully conform to physical reality in the actual marine environment. In future research, we consider introducing more complex and realistic boundary conditions to improve the practicality and accuracy of the model.

For the inverse task of pressure-release boundary detection, we have to generate the sound velocity distribution in the sound field. In that case, an inevitable problem is dealing with the speed mutation points. At these points, the speed of sound changes abruptly from the speed of sound in the liquid to 0. These points are often non-differentiable, which makes it challenging to optimize the network using the gradient descent algorithm. However, these points are crucial for predicting the pressure-release boundary structure. We propose an optimized solution to overcome this problem and generate better sound velocity distribution.

We first employ the maximum depth of the water body and multiply it by a distribution within the range of 0 to 1 to derive the depth distribution across the entire aquatic area. This distribution is generated by applying the Sigmoid activation function, which naturally maps any real-valued input to a range between 0 and 1, effectively modeling the varying depths smoothly and gradually. By leveraging the Sigmoid's characteristic S-shaped curve, we can capture the intricacies of depth variation, with shallower regions approaching 0 and deeper ones tending towards 1, thereby facilitating accurate representation of the underwater landscape for further analysis or visualization purposes.

Similarly, we use the sigmoid function to generate the distribution of the entire sound field. For example, the experiment detects the water environment below 60m, and the maximum detection depth of the water is 128m. The solution process can be expressed as Eq.(30), where $\bar{D}$ is the output tensor by the neural network for judging the depth.

$$Depth_0 = 60 + 68\text{sigmoid}(\bar{D}) \quad (30)$$

The neural network first gives a predicted depth distribution $Depth_0$. We use a one-dimensional fixed convolution to smooth it as in Eq.(31).

$$Depth = \left[\frac{1}{4} \quad \frac{1}{2} \quad \frac{1}{4}\right] \times Depth_0 \quad (31)$$

We apply Eq.(32) to obtain the depth distribution of the sound field in the domain. This function can control the mutation range within a smaller range and reasonably approximate the sound speed mutation phenomenon on both sides of the pressure release boundary. More importantly, generating this distribution is completely differentiable, so it is beneficial for the convergence of the neural network. Among them, the parameter n can be adjusted according to the scale of the sound field to be solved. In experiments, it is generally taken as -0.5 . Unreasonable settings may lead to gradient explosion.

$$Speed_0[x, y] = 1500 - \frac{1500}{1 + e^{n(y-Depth(x))}} \quad (32)$$

The initial sound speed value $Speed_0$ obtained by the above formula is then smoothed by a two-dimensional fixed convolution as shown in Eq.(33) to get the final sound speed distribution $Speed$. The pressure-release boundary configuration is fitted by inversely obtaining the sound velocity.

$$Speed = \begin{bmatrix} \frac{1}{64} & \frac{1}{64} & \frac{1}{64} & \frac{1}{64} & \frac{1}{64} \\ \frac{1}{64} & \frac{1}{32} & \frac{1}{32} & \frac{1}{32} & \frac{1}{64} \\ \frac{1}{64} & \frac{1}{32} & \frac{1}{2} & \frac{1}{32} & \frac{1}{64} \\ \frac{1}{64} & \frac{1}{32} & \frac{1}{32} & \frac{1}{32} & \frac{1}{64} \\ \frac{1}{64} & \frac{1}{64} & \frac{1}{64} & \frac{1}{64} & \frac{1}{64} \end{bmatrix} \times Speed_0 \quad (33)$$

Multi-obstacle Detection Problem. Similar to the pressure-release boundary detection task, the input is consistent, and the distribution of multiple obstacles is also fitted by outputting the sound velocity distribution. As shown in FIG. 4(c), we randomly place several sound sources in the sound field, evenly place several sensors, and perform reverse detection based on the data of these sensors to estimate the sound velocity distribution in the sound field to detect the location of obstacles. All boundaries are non-reflecting boundaries. The specific information of the cases used in the experiment is shown in Table VI. The obstacles referred to in this experiment are all cavities, and the speed of sound is consistent with the speed of sound in air 340m/s.

We have also made significant optimizations in the realm of multi-obstacle detection. Primarily, we postulate that the sound velocity field utilized for such detections should mimic the original sound field's velocity as closely as possible. We envision devising a metric to guide our approach to achieve this. However, avoiding overfitting the sound velocity is crucial, as it might obscure the distinct variations in velocity distributions caused by obstacles. Thus, we introduce a hyperparameter variable α within the sound velocity loss term. This variable is formulated as Eq.(34) based on the mean squared deviation between the sound velocities of the unobstructed sound field and those of the sound field with obstacles. We append it as a multiplicative factor to the loss function, ensuring our desired objective is met. Then, the corresponding loss function can be rewritten as Eq.(35). Among them, $\tilde{c}_{max}$ refers to the maximum value in $\tilde{c}$. $\tilde{c}_{min}$ refers to the minimum value in $\tilde{c}$. In the experiment, $(\tilde{c}_{max} - \tilde{c}_{min})$ is fixed to 1160, which is the difference between the speed of sound in air 340m/s and the speed of sound in water 1500m/s.

$$\alpha = 1 + \frac{MSE(\hat{c}, \tilde{c})}{(\tilde{c}_{max} - \tilde{c}_{min})^2} \quad (34)$$

$$Loss^{Dis} = \alpha(\Omega^D(Loss_{filed}^D + Loss_{Speed}^D) + Loss_{ref}^D) \quad (35)$$

In addition, when the neural network outputs the estimated sound speed distribution $\hat{c}$ in the two-dimensional plane, we will also smooth the sound speed distribution according to Eq.(33) to make the distribution more similar to the actual distribution. We fit the multi-obstacle distribution by inverting the obtained sound speed distribution.

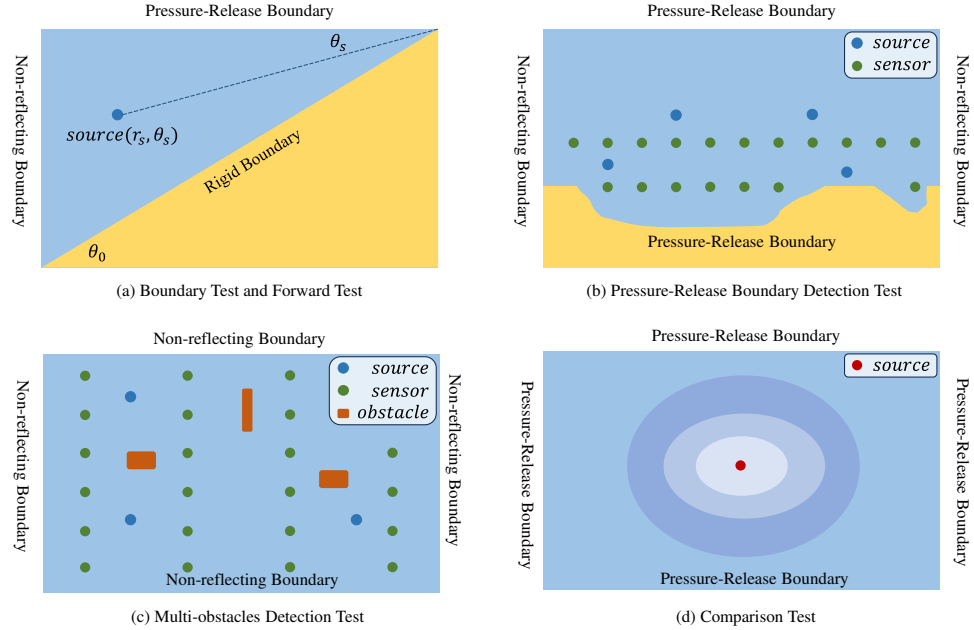


FIG. 4. Schematic diagram of the case used in the experiment. (a) A 2D wedge-shaped model with a pressure-release sea surface, rigid bottom, and non-reflecting left boundary, featuring a linear sound source at (r_s, θ_s) . The wedge angle is θ_0 , used for boundary and forward solution testing. (b) Schematic diagram of Pressure-Release Boundary detection using random sound sources and sensors. The Pressure-Release Boundary is estimated by the sound velocity distribution, with upper and lower boundaries of the Pressure-Release Boundary, and left and right boundaries that are not reflective. The contour of the lower boundary is the predicted target. (c) Multi-obstacle detection schematic with random sound sources and sensors, detecting obstacle locations by estimating sound speed distribution, all non-reflecting boundaries. (d) Comparison test case with all pressure-release boundaries, using an initial state distribution to calculate the sound field over time without specifying a sound source.

B. Platforms and Hyperparameters

The generator uses one convolutional layer with a kernel size of 3×3 , a stride of 1, and a padding of 1. For the forward problem, the maximum number of training epochs $Epoch_{max}^0$ for the first time step of the generator is 30000, and the maximum number of training epochs $Epoch_{max}$ for other time steps is 3000. For the inverse problem, the maximum number of training epochs $Epoch_{max}^0$ for the first time step of the generator is 3000, and the maximum number of training epochs $Epoch_{max}$ for other time steps is 300. The threshold ϵ for the forward problem is $1e^{-4}$, and the threshold ϵ for the inverse problem is 1. The learning rate of the generator is $1e^{-3}$.

The discriminator consists of three layers of fully connected networks, each consisting of 64 feature units. The activation function used between linear layers is $\tanh$. For the reverse task, the two networks are trained alternately 50 times. The number of epochs for a single discriminator training is 200 for

the first training and 100 after alternating training. The optimization algorithm is the *AdamW* algorithm⁴³. The learning rate of the discriminator is $1e^{-3}$.

The GPU configuration used in the experiment is NVIDIA GeForce RTX 4090. All codes are implemented based on Pytorch⁴⁴. The water sound speed is $1500m/s$, and the sound speed in cavities is $340m/s$. Other unknown parameters are determined according to the specific task.

C. Results and Discussion of Forward Solving tests

This section presents and discusses the experimental results of two forward-solving tasks: boundary condition testing and forward-solving task with analytical solution.

Test	Type	Input	Output	Diagram	Case Number	Case Information	Results Analysis
Boundary Conditions Test	Forward	Distribution of Sound Speed, Initial State, Source Location, Sound Wave Form	Distribution of sound field ΔP	FIG. 4(a)	1	Table III Section III A 1 Section III C 1	Section III C 1
Forward Solving Test	Forward	Distribution of Sound Speed, Initial State, Source Location, Sound Wave Form	Distribution of sound field ΔP	FIG. 4(a)	2-4	Table III Section III A 1 Section III C 2	Section III C 2
Pressure-Release Boundary Detection Task	Inverse	Sampling Point Location, Sampling Point Data, Source Location, Sound Wave Form	Distribution of sound speed $\hat{c}$	FIG. 4(b)	5-8	Table V Section III A 2 Section III D 1	Section III D 1
Multi-obstacle Detection Test	Inverse	Sampling Point Location, Sampling Point Data, Source Location, Sound Wave Form	Distribution of sound speed $\hat{c}$	FIG. 4(c)	9-12	Table VI Section III A 2 Section III D 2	Section III D 2
Comparison Test	Forward	Distribution of Sound Speed, Initial State	Distribution of sound field ΔP	FIG. 4(d)	13	Section III A 1 Section III E	Section III E

TABLE II. A list of key parameters for each tasks.

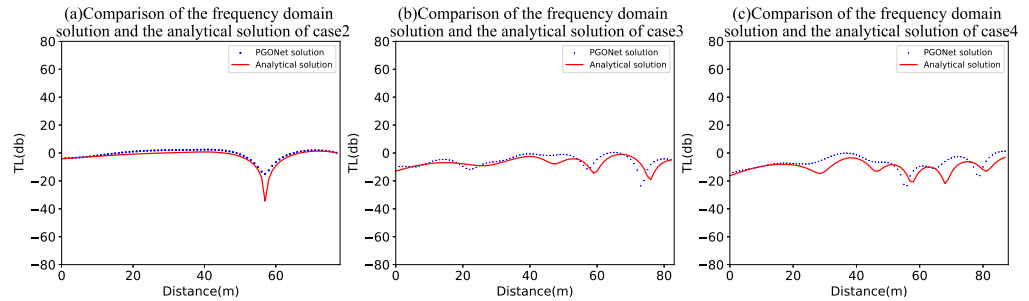


FIG. 5. This figure shows the time domain solution of Case 2-4 calculated by the solver after it is converted into the frequency domain solution¹² and compared with the analytical solution.

TABLE III. Introduction to all test cases used in Boundary Test and forward-solving test.

No	Sound Source Coordinates(m)	Form of Sound Wave	Depth of sample lines
1	[120,120]	0.01s Square Wave	-
2	[40,40]	32Hz Sine Wave	50m Depth Line
3	[40,40]	64Hz Sine Wave	45m Depth Line
4	[30,30]	64Hz Sine Wave	40m Depth Line

TABLE IV. Error analysis table for forward-solving test cases 2-4. The average absolute error is calculated by $\frac{1}{n} \sum |\Delta P - P_{True}|$, where ΔP is the PGONet calculated value and P_{True} is the true value, n is the number of grids.

No	Average Absolute Error(dB)	Number of Sampling Points	Comparison Figure
2	3.27	77	FIG. 5(a)
3	4.52	82	FIG. 5(b)
4	5.27	87	FIG. 5(c)

1. Boundary Conditions Tests

We tested three boundary conditions, which are the non-reflecting boundary, the rigid boundary, and the pressure-release boundary. We use the case 1 shown in FIG. 4(a), Table II and Table III, set the sound source at $(120m, 120m)$. This is a wedge model that includes three boundary conditions. The left side is a non-reflecting boundary, the top is a pressure-release boundary, and the bottom is a rigid boundary. We adjusted the sound speed from $1500m/s$. We take the sound field $0.01s$ after the sound source propagates as the initial state ($t = 0.00s$). The grid size is 128×128 , each grid size is $3m \times 3m$, and the sound source applies a square wave with a duration of $0.01s$ and a maximum value of 1500 . The time step interval is $\frac{1}{1024}s$. This test has 128 time steps.

The experimental results are shown in FIG. 6. The absorption boundary condition on the left can absorb sound waves well without generating additional reflections. Evident reflections will be generated under the hard reflection boundary condition on the hypotenuse side of the wedge. Under the pressure-release boundary condition on the top, apparent reflections will also be generated, and the numerical signs will be reversed. The FDM simulation results in PGONet are highly consistent on the three boundary conditions, which shows that the model performs well in the boundary condition test.

2. Forward Solving Test

Initially, we conducted forward solution experiments by solving the forward time term in $128m \times 128m$ waters domain and compared the solution results with the analytical solution. The size of the grid in the current test is $1m$. We use three cases in this task, numbered cases 2-4. Each case's sound source position, sound source form, and sampling line position are shown in Table III. The following case uses a sine wave, which is the same and will not be explained separately. The time step length is $\frac{1}{16384}s$, totaling 4096 time steps. We transform the time domain data of the last 2048 time steps into the frequency domain and compare it with the calculated analytical solution⁵. The boundary conditions of the case are set the same as those in FIG. 4(a) and Table III. The sound source amplitude is 1500 .

The comparison diagrams of Case 2-4 between the propagation predicted by PGONet and the FDM calculation are shown in FIG. 7, 8, and 9, respectively. We can find that the three cases are roughly consistent. The quantitative experimental results are shown in FIG. 5 and Table IV. According to FIG. 5, compared with the corresponding analytical solution, it can be found that the numerical range calculated by PGONet is consistent with the analytical solution. We also calculated the average absolute error (calculated as $\frac{1}{n} \sum |\Delta P - P_{true}|$, where ΔP is the PGONet calculated value and P_{true} is the actual value, n is the number of grids). As shown in Table IV and found that the average absolute error can be limited to within $5.5dB$. PGONet achieves reasonable simulation results for these situations on coarse grids. How-

ever, experiments also show that the accuracy of the simulation results decreases as the frequency of the sound source increases, which is due to the accuracy limitation of the iterative model.

D. Results and Discussion of Inverse Solving tests

This section presents and discusses experimental results on two inverse problem-solving tasks: the Pressure-Release Boundary Detection Problem and the Multi-obstacle Detection Problem.

1. Pressure-Release Boundary Detection Inversion Experiment

The introduction of the pressure-release boundary detection test case is shown in FIG. 4(b) and Table II. We applied several sensors in a water area of $128m \times 128m$. The sensors were randomly selected and placed. We released several sound sources in each case. They are all sine waves with a frequency of 25 Hz and a maximum value of 1500 . The time step is $\frac{1}{4096}s$, with 256 timesteps. For the convenience of the experiment, we assume that the lower boundary to be predicted is the pressure-release boundary. We refer to the description in section III A 2 and detect the structure at $60m$ underwater. The detailed information of each case is shown in Table V, and the schematic diagram is shown in FIG. 10(a)-(d). FIG. 10(a)-(d) mark the yellow sensor locations, bright red sound source points, and blue soundless structures under the pressure-release boundary.

We thoroughly evaluate and examine our model in detail in four different scenarios. The results of these exhaustive experiments in FIG. 10 show that our model can reasonably predict the pressure-release boundary profile. The general trend is consistent with the contour. Our model performs well by relying solely on the physics-driven neural network without additional training data. This demonstrates the efficiency of our proposed model on this task.

TABLE V. Table of all test cases used in the pressure-release boundary detection.

No	Number of Sensors	Location of Sources (m)	Diagram of the case
5	39	[85,20], [85,40],[100,60] [65,100],[85,120]	FIG. 10(a)
6	27	[75,10], [75,40],[75,70] [75,100],[75,120]	FIG. 10(b)
7	27	[70,10], [80,30],[80,60] [80,90],[65,110]	FIG. 10(c)
8	27	[100,12], [75,30],[70,56] [70,76],[75,106], [100,120]	FIG. 10(d)

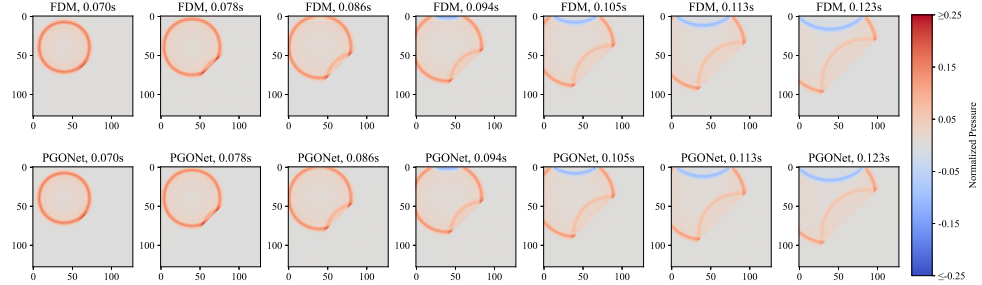


FIG. 6. Boundary comparison of PGONet with Finite Difference Method (FDM) on case 1. The case boundary condition settings are shown in FIG. 4(a), Table II and Table III. The upper part is the distribution calculated by the Finite Difference Method (FDM), and PGONet calculates the lower part. The sound pressure is normalized to the range of $[-1, 1]$.

TABLE VI. Table of all test cases used for multi-obstacle detection.

No	Number of Obstacles	Location of Obstacles (m)	Diagram of the case
9	4	[13:20,33:35], [24:28,102:106], [86:92,86:90], [95:102,25:32]	FIG. 11(a)
10	2	[22:27,103:109], [98:105,16:19]	FIG. 11(b)
11	3	[22:27,33:39], [48:55,61:69], [90:98,91:96]	FIG. 11(c)
12	1	[42:47,58:61]	FIG. 11(d)

2. Multi-obstacle Detection Inversion Experiment

The introduction of the multi-obstacle detection test is shown in FIG. 4(c) and Table II. We have already described it in detail in Section III A 2. We apply 64 sensors in a water area of $128m \times 128m$. The sensors are evenly placed in a 8×8 layout, and one is placed every $15m$ in the horizontal and vertical directions. The four boundaries in the water area are pressure-release boundaries. We release five sound sources at $(20m, 20m)$, $(20m, 100m)$, $(64m, 64m)$, $(20m, 100m)$ and $(100m, 100m)$ at the same time. They are all sine waves with a frequency of 25 Hz and a maximum value of 1500. The time step length is $\frac{1}{4096}s$, with 256 time steps. For the convenience of experiments, we assume that the speed of sound at the obstacle's position is 340m/s. Based on this, the obstacles existing in the water area are estimated. We designed four cases for this problem, and the obstacle position distribution and sound speed in the four cases are shown in Table VI and FIG. 11(a)-(d). The locations of multiple obstacles within the water area are marked in blue in FIG. 11(a)-(d). All boundaries are non-reflecting boundaries.

As shown in the FIG. 11, we have comprehensively evaluated our proposed model, rigorously testing the four scenarios outlined previously. The results of this rigorous testing demonstrate the excellent ability of our model to accurately predict and estimate the positions of multiple obstacles in a water environment due to the anisotropy of the speed of sound.

This achievement is achieved without any prior training or learning phase tailored explicitly to the model, relying only on partially acquired data samples and physical laws, highlighting our approach's effectiveness.

E. Results and Discussion of Comparison Test

In this section, we compare the performance of the PGONet generator with the residual connection module, FDM, PINN¹⁶, UNet⁴⁵, and PGONet without the residual connection module in the forward solution, focusing on the accuracy and speed of the solution. Both PINN and UNet have 5 layers, and the number of neurons in a single layer (or hidden layer channels) is 64. The activation function of PINN is Tanh, and the activation function of UNet is ReLU. The learning rate is $1e^{-3}$, and it decreases to the original 0.98 every 5000 epochs, and the number of training epochs is 50000. For other detailed parameters, please refer to the code.

We use Case 13 in this test. The configuration of Case 13 is shown in FIG. 4(d). We have given a brief introduction in Section III A 1 and Table II. In this case, the initial state and sound velocity distribution are given, and it is required to calculate the sound field distribution within a given time range. The initial state distribution of the sound field obeys Eq.(29). The sound field size, in this case, is 128×128 , and the grid size is $100m$. For non-grid-dependent methods, boundary condition constraints and initial condition approximations are implemented by introducing a loss term multiplied by a weight of $1e3$. For the grid-dependent method, both initial and boundary conditions are enforced. When calculating the residuals, we uniformly select 127008 ($32 \times 63 \times 63$) points in the time and space domain. The water sound speed is 1500m/s. The test time step of this case is $\frac{1}{25}s$. There are 64 time steps in total. The sound source applies a square wave with a duration of $\frac{1}{25}s$ and a maximum value of 1000. The coordinates of the sound source on the grid are (64, 64).

We compare the propagation effects of the finite differ-

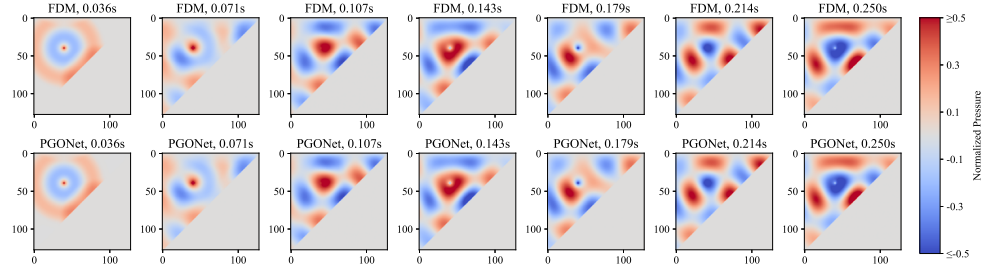


FIG. 7. The figure shows the propagation from 0.036s to 0.250s in case 2. The upper part is the distribution calculated by the finite difference method (FDM), and PGONet calculates the lower part. The sound pressure is normalized to the range of $[-1, 1]$.

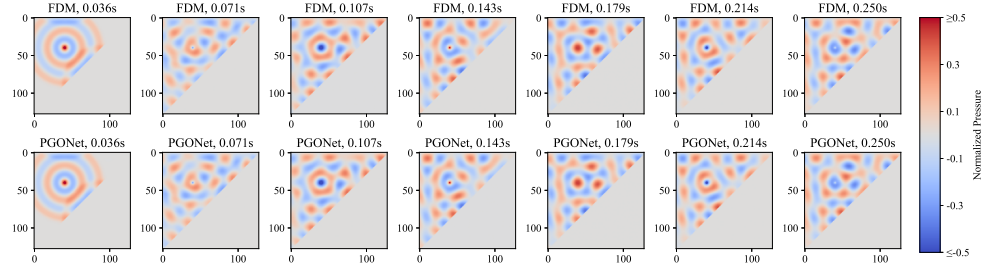


FIG. 8. The figure shows the propagation from 0.036s to 0.250s in case 3. The upper part is the distribution calculated by the finite difference method (FDM), and PGONet calculates the lower part. The sound pressure is normalized to the range of $[-1, 1]$.

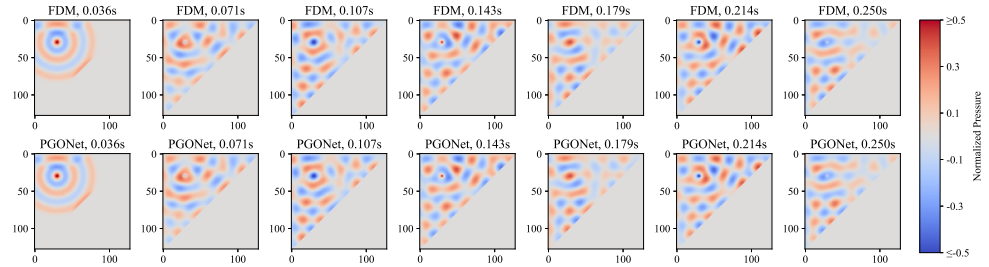


FIG. 9. The figure shows the propagation from 0.035s to 0.250s in case 4. The upper part is the distribution calculated by the finite difference method (FDM), and PGONet calculates the lower part. The sound pressure is normalized to the range of $[-1, 1]$.

ence method (FDM), PINN, PGONet with residual connection module, PGONet without residual connection module, and UNet in Case 13, as shown in FIG. 12 and FIG. 13. First, the simulation results of these four networks show apparent signs of convergence in appearance. The forward sound field prediction given by PGONet with the residual connection module is highly consistent with the finite difference method (FDM) calculation results. The forward sound field prediction given by PGONet without the residual connection module is

also consistent with the finite difference method (FDM) calculation results. UNet fits the propagation process, but its error is more evident than PGONet. PINN can only give a basic propagation outline, and the propagation results show apparent errors. This shows that PGONet can achieve better results in underwater acoustic application scenarios than other models and has superior accuracy performance.

Finally, we list the time running efficiency of each model in Table VII. In terms of running time, the total running time

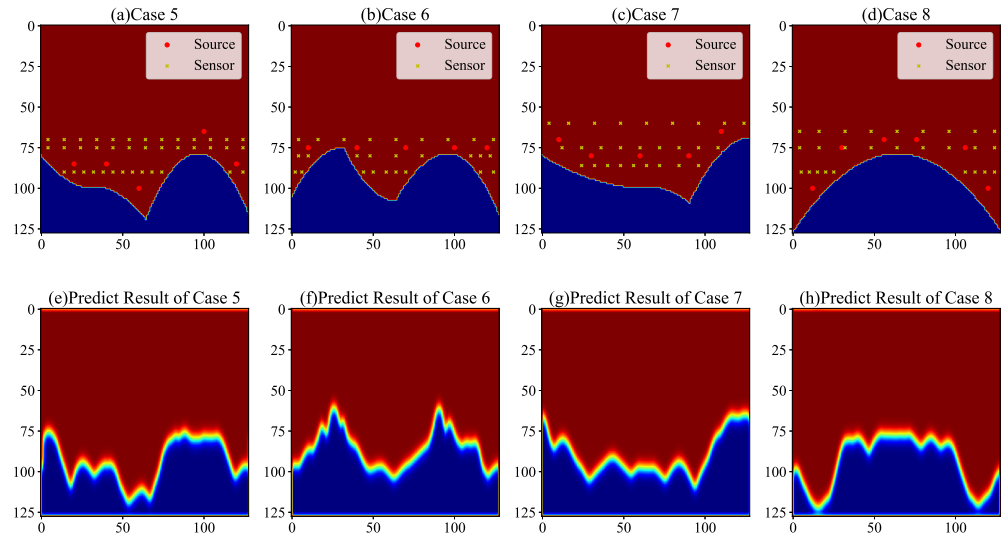


FIG. 10. Comparison of the prediction results given by PGONet and the actual configuration of the pressure-release boundary detection cases 5-8. In sub-graphs (a)-(d), the blue closed area depicts the contour, the yellow dot indicates the sensor location, and the red dot indicates the sound source location.

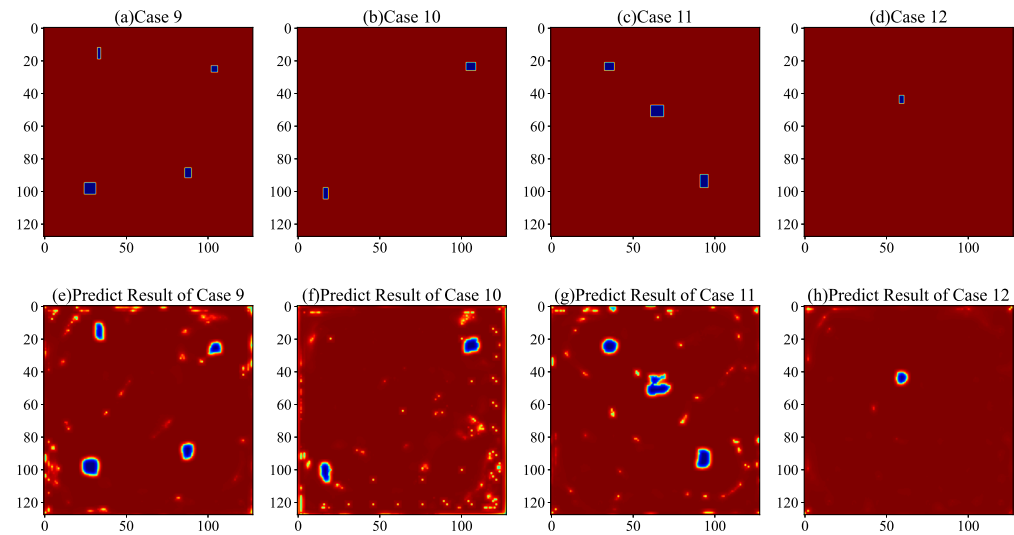


FIG. 11. Comparison of the prediction results given by PGONet and the results of multi-obstacle detection cases 9-12. The positions of multiple obstacles are depicted in blue closed areas in sub-images (a)-(d). The test results in sub-graphs (e)-(h) show that PGONet can accurately reflect the multi-obstacle detection.

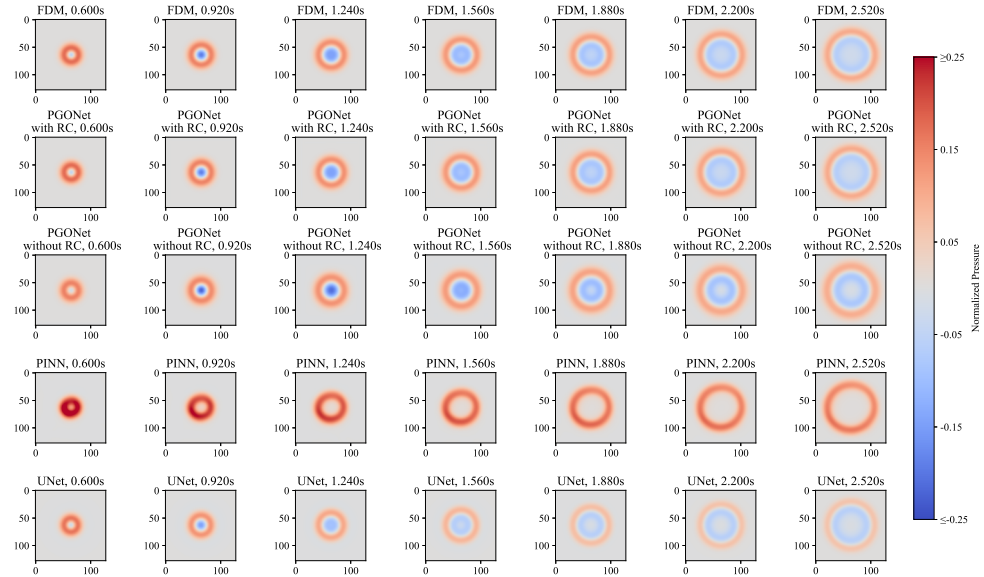


FIG. 12. Comparison of the propagation results of the finite difference method (FDM), PGONet's generator with Residual Connection (RC) module, PGONet's generator without residual connection module, PINN and UNet from 0.600s to 2.520s in Case 13. The results from the first row to the fifth row are the simulation results of FDM, PGONet's generator with residual connection module, PGONet's generator without residual connection module, PINN, and UNet. The sound pressure is normalized to the range of $[-1, 1]$.

of the PGONet generator is 51.38s, which is 28.66 and 50.69 times faster than the total running time of the baseline UNet and PINN, respectively. The running time of each round of the PGONet generator is 0.0016s, which is 18.44 and 32.56 times faster than the total running time of the baseline UNet and PINN, respectively. The PGONet generator without the residual connection module has a single-round running speedup ratio of 1.45 relative to the PGONet with the residual connection module, but in terms of total running time, since the residual connection module sets a stopping standard, the early stopping will make the total running time faster than it by 4.86. This shows that our proposed model saves more time when solving problems.

TABLE VII. Comparison of the running time of PINN, UNet, and PGONet generators with and without residual connection module on performance test case 13. RC stands for residual connection module.

Model	Total Time(s)	Time Per Epoch(s)
PINN	2604.69	0.0521
UNet	1472.74	0.0295
PGONet without RC	249.89	0.0011
PGONet with RC	51.38	0.0016

IV. CONCLUSION

Due to the complexity and variability of the marine environment, ocean exploration presents significant challenges, with ocean acoustics playing a crucial role in this field. Traditional numerical algorithms have been the primary focus of research in ocean acoustics, but recent advancements in AI have sparked new inspiration for researchers.

We designed and optimized a physics-informed generative adversarial network PGONet for solving forward and inverse problems in ocean acoustics. The network structure was designed and optimized based on aspects such as network structure design, hard constraint boundary conditions, adaptive weighting loss function, and inverse task optimization. The efficiency and accuracy of PGONet in forward solution were verified through boundary condition experiments, forward solution tests, and comparison tests. In addition, we also tested the performance of PGONet in multi-obstacle detection and pressure-release boundary detection inverse solution tasks. The results show that it has good results in solving inverse problems in ocean acoustics.

Although our research has made progress, there are still some limitations to our work. First, PGONet still processes grid-structured data, which depends on factors such as grid quality. Simulation on more complex grids is still a topic to be studied. Second, our work is still on a single GPU, which

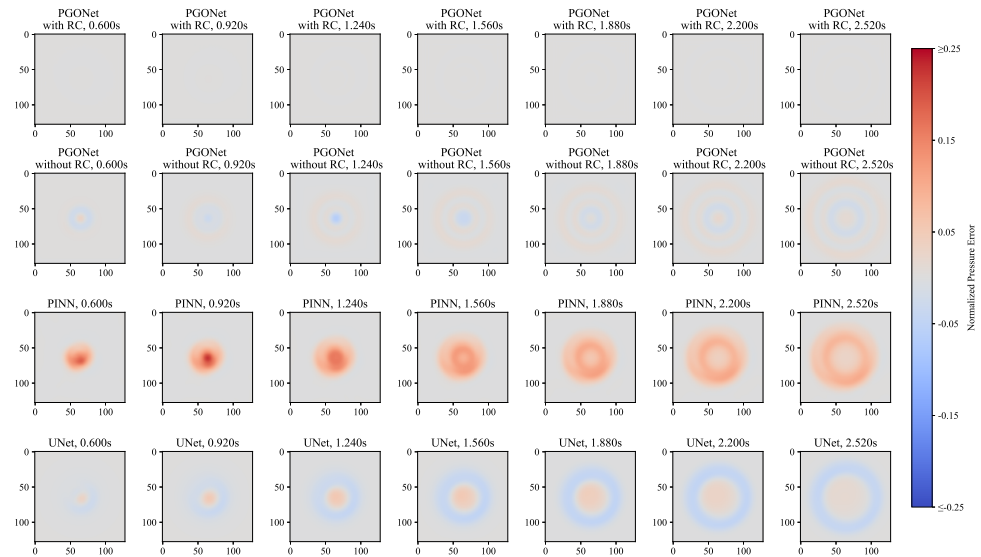


FIG. 13. Comparison of propagation error of the finite difference method (FDM), PGONet's generator with Residual Connection (RC) module, PGONet's generator without residual connection module and PINN from 0.600s to 2.520s in Case 13. The results from the first to fourth rows are the errors between the simulation results of the PGONet generator with a residual connection module, the PGONet generator without a residual connection module, PINN, and UNet and the FDM simulation results. The sound pressure error is normalized to the range of $[-1, 1]$.

is insufficient for large-scale ocean problems. Finally, combined with our priority understanding of academic frontiers, we believe that PGONet still has room for further improvement in the prediction accuracy of long-time series tasks and the accuracy of solving inverse problems.

In future research, we will focus on enhancing the integration of physical information into widely used neural network architectures, aiming to improve their learning and training effects. Specifically, we will develop a large-scale optimized version of PGONet to address more complex hydroacoustic forward and inverse problems. In addition, we plan to expand our framework to incorporate fluid boundary dynamics, such as impedance and layered medium effects, to improve the model's realism and applicability to real marine environments. This will involve validating the model against experimental data from fluid boundary scenarios and exploring mixed situations where rigid and fluid boundary assumptions coexist, bridging the gap between theoretical models and practical applications. Furthermore, we will prioritize a deeper incorporation of fluid physics, particularly in areas such as boundary condition modeling, wave propagation in stratified media, and energy dissipation mechanisms, to enhance the model's understanding of underwater acoustic phenomena and refine its application to real-world problems.

ACKNOWLEDGEMENT

The work is supported by National Key Research and Development Program of China under Grant No. (No.2021YFB0300101, No.2023YFA1011704) and the National Natural Science Foundation of China (No.12102468, No.12002380) and the National University of Defense Technology Foundation (No.ZK21-02, No.ZK20-52).

DATA AVAILABILITY STATEMENT

The code and data that support the findings of this study are openly available in <https://github.com/DongFengZero/PGONet>.

¹F. B. Jensen, W. A. Kuperman, M. B. Porter, H. Schmidt, and A. Tolstoy, *Computational ocean acoustics*, Vol. 2011 (Springer, 2011).

²N. Gao and K. Lu, "An underwater metamaterial for broadband acoustic absorption at low frequency," *Applied Acoustics* **169**, 107500 (2020).

³H. Sagen, M. Dzieciuch, and P. Worcester, "Monitoring the ocean climate with acoustics," *The Journal of the Acoustical Society of America* **153**, A25–A25 (2023).

⁴A. Mooney, L. Di Iorio, M. Lammers, T.-H. Lin, S. Nedelec, M. Parsons, C. Radford, E. Urban, and J. Stanley, "Listening forward: approaching marine biodiversity assessments using acoustic methods," *Royal Society Open Science* **7**, 201287 (2020).

⁵W. Luo, C. Yang, J. Qin, and R. Zhang, "A numerically stable coupled-mode formulation for acoustic propagation in range-dependent waveguide."

- uities," *Science China Physics, Mechanics and Astronomy* **55** (2012), 10.1007/s11433-012-4666-0.
- ⁶B. J. DeCourcy and T. F. Duda, "A coupled mode model for omnidirectional three-dimensional underwater sound propagation," *The Journal of the Acoustical Society of America* **148**, 51–62 (2020).
- ⁷P. S. Petrov, M. Ehrhardt, A. G. Tyshchenko, and P. N. Petrov, "Wide-angle mode parabolic equations for the modelling of horizontal refraction in underwater acoustics and their numerical solution on unbounded domains," *Journal of Sound and Vibration* **484**, 115526 (2020).
- ⁸T. He, J. Liu, S. Ye, X. Qing, and S. Mo, "A novel model order reduction technique for solving horizontal refraction equations in the modeling of three-dimensional underwater acoustic propagation," *Journal of Sound and Vibration* **591**, 118617 (2024).
- ⁹M. B. Porter, "Beam tracing for two-and three-dimensional problems in ocean acoustics," *The Journal of the Acoustical Society of America* **146**, 2016–2029 (2019).
- ¹⁰Y.-T. Lin, "Three-dimensional boundary fitted parabolic-equation model of underwater sound propagation," *The Journal of the Acoustical Society of America* **146**, 2058–2067 (2019).
- ¹¹B.-H. Jung and T. Sarkar, "Solving time domain helmholtz wave equation with mod-fdm," *Progress In Electromagnetics Research* **79** (2008), 10.2528/PIER07102802.
- ¹²R. Xia, X.-W. Guo, C. Li, and J. Liu, "Direct numerical simulation of acoustic wave propagation in ocean waveguides using a parallel finite volume solver," *Ocean Engineering* **281**, 114894 (2023).
- ¹³A. Bottero, P. Cristini, D. Komatitsch, and M. Asch, "An axisymmetric time-domain spectral-element method for full-wave simulations: Application to ocean acoustics," *The journal of the acoustical society of America* **140**, 3520–3530 (2016).
- ¹⁴C. Li, B. K. Campbell, Y. Liu, and D. K. Yue, "An $\mathcal{O}(n \log n)$ multi-layer boundary element method for direct computation of sound propagation in shallow water environments," *Proposed Journal Article*, unpublished **2018** (2018).
- ¹⁵T. He, S. Mo, E. Fang, M. Wang, and R. Zhang, "Modeling three-dimensional underwater acoustic propagation over multi-layered fluid seabeds using the equivalent source method," *The Journal of the Acoustical Society of America* **150**, 2854–2864 (2021).
- ¹⁶M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics* **378** (2018), 10.1016/j.jcp.2018.10.045.
- ¹⁷L. Yang, X. Meng, and G. Karniadakis, "B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data," *Journal of Computational Physics* **425**, 109913 (2021).
- ¹⁸E. Kharazmi, Z. Zhang, and G. E. Karniadakis, "hp-vpinns: Variational physics-informed neural networks with domain decomposition," *Computer Methods in Applied Mechanics and Engineering* **374**, 113547 (2021).
- ¹⁹Q. Zhang, X.-W. Guo, X. Chen, C. Xu, and J. Liu, "Pinn-fht: A physics-informed neural network for solving fluid flow and heat transfer problems without simulation data," *International Journal of Modern Physics C* **33** (2022), 10.1142/S0129183122501662.
- ²⁰J. Yan, X. Chen, Z. Wang, E. Zhou, and J. Liu, "St-pinn: A self-training physics-informed neural network for partial differential equations," (2023), arXiv:2306.09389 [cs.LG].
- ²¹C. Xu, B. T. Cao, Y. Yuan, and G. Meschke, "Transfer learning based physics-informed neural networks for solving inverse problems in engineering structures under different loading scenarios," *Computer Methods in Applied Mechanics and Engineering* **405**, 115852 (2023).
- ²²Y. Chen, L. Lu, G. Karniadakis, and L. Negro, "Physics-informed neural networks for inverse problems in nano-optics and metamaterials," *Optics Express* **28** (2020), 10.1364/OE.384875.
- ²³K. Shukla, P. Clark di Leoni, J. Blackshire, D. Sparkman, and G. Karniadakis, "Physics-informed neural network for ultrasound nondestructive quantification of surface breaking cracks," *Journal of Nondestructive Evaluation* **39** (2020), 10.1007/s10921-020-00705-1.
- ²⁴M. Rasht-Behesht, C. Huber, K. Shukla, and G. Karniadakis, "Physics-informed neural networks (pinns) for wave propagation and full waveform inversions," *Journal of Geophysical Research: Solid Earth* **127** (2022), 10.1029/2021JB023120.
- ²⁵P. Borate, J. Rivière, C. Marone, A. Mali, D. Kifer, and P. Shokouhi, "Using a physics-informed neural network and fault zone acoustic monitoring to predict lab earthquakes," *Nature Communications* **14** (2023), 10.1038/s41467-023-39377-6.
- ²⁶Y. Zhu, N. Zabarar, P. Koutsourelakis, and P. Perdikaris, "Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data," *Journal of Computational Physics* **394** (2019), 10.1016/j.jcp.2019.05.024.
- ²⁷H. Gao, M. Zahr, and J.-X. Wang, "Physics-informed graph neural galerkin networks: A unified framework for solving pde-governed forward and inverse problems," *Computer Methods in Applied Mechanics and Engineering* **390**, 114502 (2022).
- ²⁸W. Zha, Y. Liu, Y. Wan, R. Luo, D. Li, S. Yang, and Y. Xu, "Forecasting monthly gas field production based on the cnn-lstm model," *Energy* **260**, 124889 (2022).
- ²⁹G. Dou, K. Zhao, M. Guo, and J. Mou, "Memristor-based lstm network for text classification," *Fractals* (2023), 10.1142/S0218348X23400406.
- ³⁰Y. Zhou, C. Lu, K. Chen, and X. Li, "Sea surface height anomaly prediction based on artificial intelligence," (2023) pp. 63–82.
- ³¹M. Collins and W. Kuperman, "Inverse problems in ocean acoustics," *Inverse Problems* **10**, 1023 (1999).
- ³²P. Chai, L. Hou, G. Zhang, Q. Tushar, and Y. Zou, "Generative adversarial networks in construction applications," *Automation in Construction* **159**, 105265 (2024).
- ³³L. Geng, L.-Z. Zhou, H. Shen, C.-D. He, and F. Xie, "An iteratively reweighted time-domain acoustic method for reconstructing the transient acoustic field," *Signal Processing* **210**, 109076 (2023).
- ³⁴P. Ren, C. Rao, Y. Liu, J.-X. Wang, and H. Sun, "Phycrnet: Physics-informed convolutional-recurrent network for solving spatiotemporal pdes," *Computer Methods in Applied Mechanics and Engineering* **389**, 114399 (2022).
- ³⁵N. Sukumar and A. Srivastava, "Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks," *Computer Methods in Applied Mechanics and Engineering* **389**, 114333 (2021).
- ³⁶W. Jiaji, Y. Mo, B. Izzuddin, and C.-W. Kim, "Exact dirichlet boundary physics-informed neural network epinn for solid mechanics," *Computer Methods in Applied Mechanics and Engineering* **414**, 116184 (2023).
- ³⁷L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, and S. G. Johnson, "Physics-informed neural networks with hard constraints for inverse design," *SIAM Journal on Scientific Computing* **43**, B1105–B1132 (2021).
- ³⁸H. Jeong, C. Batuwatta-Gamage, J. Bai, Y. M. Xie, C. Rathnayaka, Y. Zhou, and Y. Gu, "A complete physics-informed neural network-based framework for structural topology optimization," *Computer Methods in Applied Mechanics and Engineering* **417**, 116401 (2023).
- ³⁹S. Yang, S. Lee, and K. Yee, "Inverse design optimization framework via a two-step deep learning approach: application to a wind turbine airfoil," *Engineering with Computers* **39**, 2239–2255 (2023).
- ⁴⁰T. Guo, D. J. Lohan, R. Cang, M. Y. Ren, and J. T. Allison, "An indirect design representation for topology optimization using variational autoencoder and style transfer," in *2018 AIAA/ASCE/AHS/ASC structures, structural dynamics, and materials conference* (2018) p. 0804.
- ⁴¹M. Buckingham and A. Tolstoy, "An analytical solution for benchmark problem 1: The 'ideal' wedge," *Journal of The Acoustical Society of America - J ACOUST SOC AMER* **87**, 4 (1990).
- ⁴²L. Felsen, "Numerical solutions of two benchmark problems," *J. Acoust. Soc. Am.* **81**, S39–S40 (1987).
- ⁴³I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," (2019), arXiv:1711.05101 [cs.LG].
- ⁴⁴N. Ketkar, "Introduction to pytorch," (2017) pp. 195–208.
- ⁴⁵L. Wang, J. Li, S. Chen, Z. Fan, Z. Zeng, and Y. Liu, "Finite difference-embedded unet for solving transcranial ultrasound frequency-domain wavefield," *The Journal of the Acoustical Society of America* **155**, 2257–2269 (2024).